

# RHYNK

"Every Chat Has a Rhythm."

## Database Design & Entity-Relationship Deep-Dive

**Type:** Engineering R&D Document — Phase 1 Foundation  
**Scope:** PostgreSQL (Prisma) · MongoDB · Redis — schema, ERDs, rationale, scaling path  
**Date:** July 2026  
**Companion PRD:** Rhynk PRD v2.2  
**Author context:** Nirvikar — fullstack, first-principles engineering approach

# Table of Contents

---

- 1 Why three databases — the polyglot-persistence decision
- 2 PostgreSQL — entity-relationship diagrams
- 3 PostgreSQL — full table specifications & design rationale
- 4 MongoDB — messages & stories schema design
- 5 Redis — key design & TTL strategy
- 6 Indexing, cascade rules & query-path analysis
- 7 Scaling path — from 500 to 25,000 DAU and beyond
- 8 GDPR / DPDPA compliance in the schema
- 9 Does Prisma auto-generate ER diagrams? — direct answer
- 10 Full `schema.prisma` reference

# 1. Why three databases — the polyglot-persistence decision

Rhynk's data has three fundamentally different shapes, and forcing them into one engine would mean compromising on all three. The R&D question isn't "which database is best" — it's "what does each write pattern actually need."

| Store      | What lives here                                                                                             | Why this store, specifically                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL | Users, conversations, membership, music room metadata, songs, playlists, notifications, call logs           | This is <b>relational, low-to-medium write volume, high-integrity</b> data. "Is this user a member of this conversation" is a join-heavy question asked on almost every request — a relational engine with real foreign keys and transactions is the correct tool. Prisma migrations give you a reviewable, versioned schema history, which matters once more than one engineer touches this.                                                                                                                                                                                                                                                                          |
| MongoDB    | Messages, stories/status                                                                                    | This is the <b>highest write-volume collection in the whole system</b> (PRD §2.2 projects 3M messages/day by month 6) and each message document has a genuinely variable shape — text vs. image vs. song-share vs. file all carry different <code>content</code> fields, plus a growing <code>reactions</code> array and <code>read_by</code> array that mutate independently of the message body. Modelling that in a rigid relational table means either a wide sparse table or 4 extra join tables purely to append an emoji reaction. A document store lets the message be one atomic, self-contained unit that's cheap to write and cheap to read back in a feed. |
| Redis      | Presence, live room playback state, room queue, typing indicators, rate limits, OTP, refresh-token sessions | Everything here shares one property: <b>it mutates multiple times per second per active unit, and losing it on a crash is fine</b> because it's either re-derivable (presence, typing) or intentionally short-lived (OTP, rate limit windows). Writing a room's <code>position_ms</code> to Postgres on every playback tick would mean the primary relational database absorbs a write load that has nothing to do with its actual job of protecting durable state. Redis's TTL and atomic <code>SETNX</code> primitives also directly solve two hard problems the PRD calls out: OTP expiry and DJ-election / refresh-token race conditions (§8.4, §15).              |

**R&D framing:** a useful test for "which store does this field belong in" is: *if this value changed 10 times a second, would anyone notice if we lost the last value on a server restart?* If yes → Redis. If the value needs to survive forever and be queried relationally → Postgres. If it's an append-heavy, variably-shaped, per-conversation timeline → MongoDB.

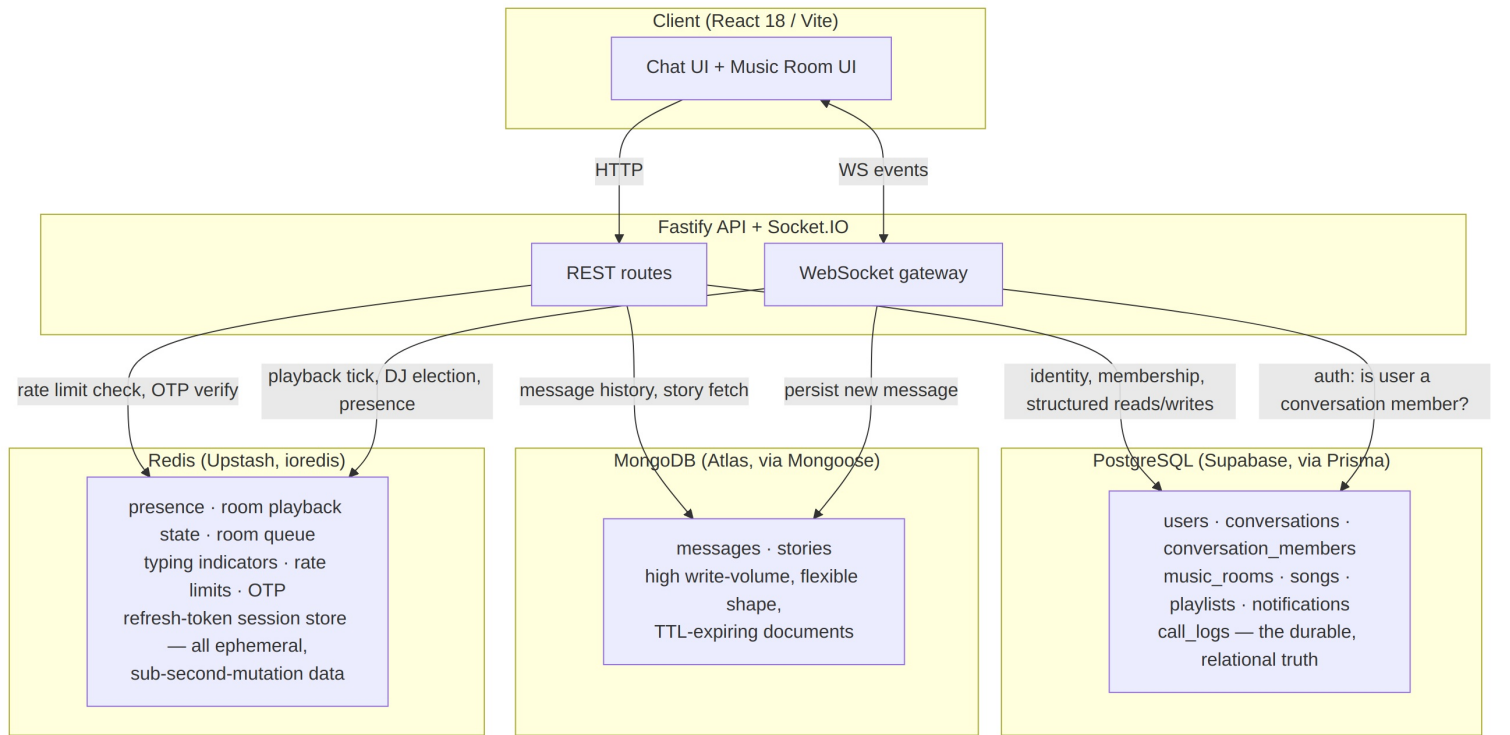


Figure 1 — how a single message-send / room-tick touches each store

## 2. PostgreSQL — entity-relationship diagrams

Split into two diagrams for readability: the **identity & messaging structure** core, and the **music domain**. Both share the `users` and `conversations` tables as their anchor points.

### 2.1 Core: identity, conversations, membership, notifications, calls

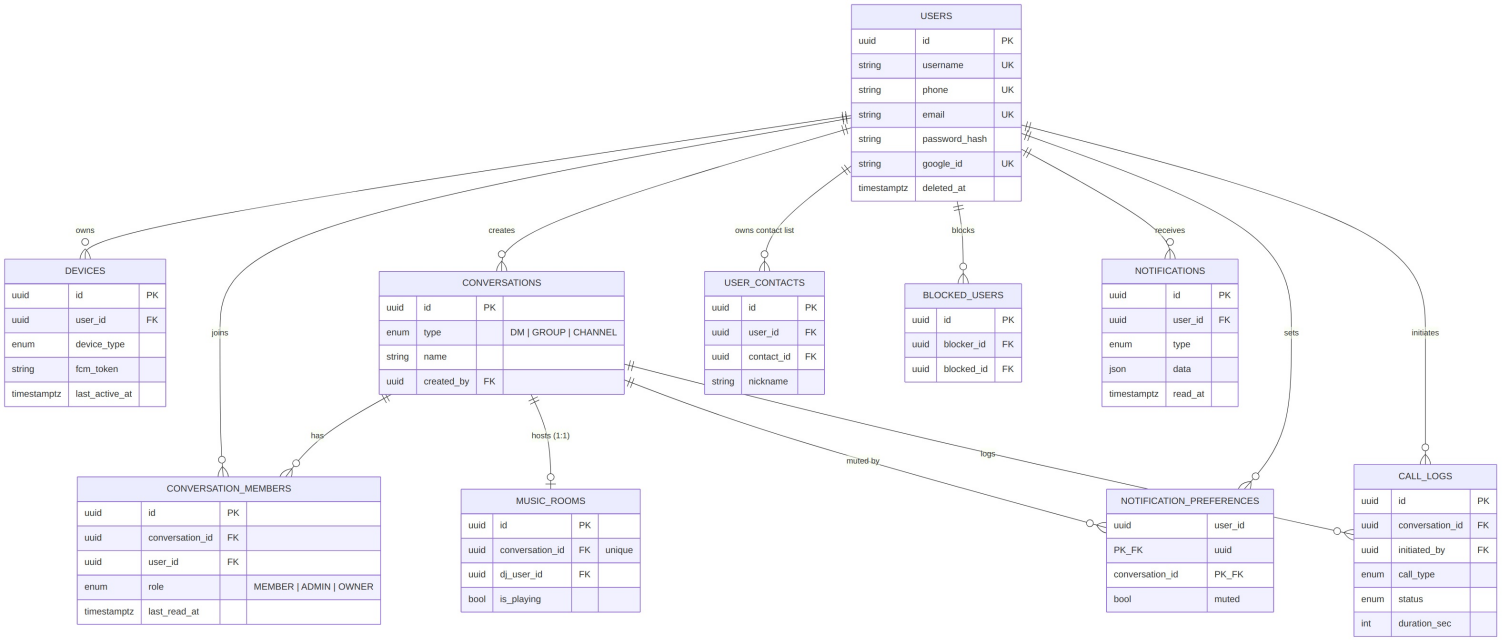


Figure 2 — core relational ERD. PK = primary key, FK = foreign key, UK = unique key.

2.2 Music domain: rooms, songs, playlists, likes

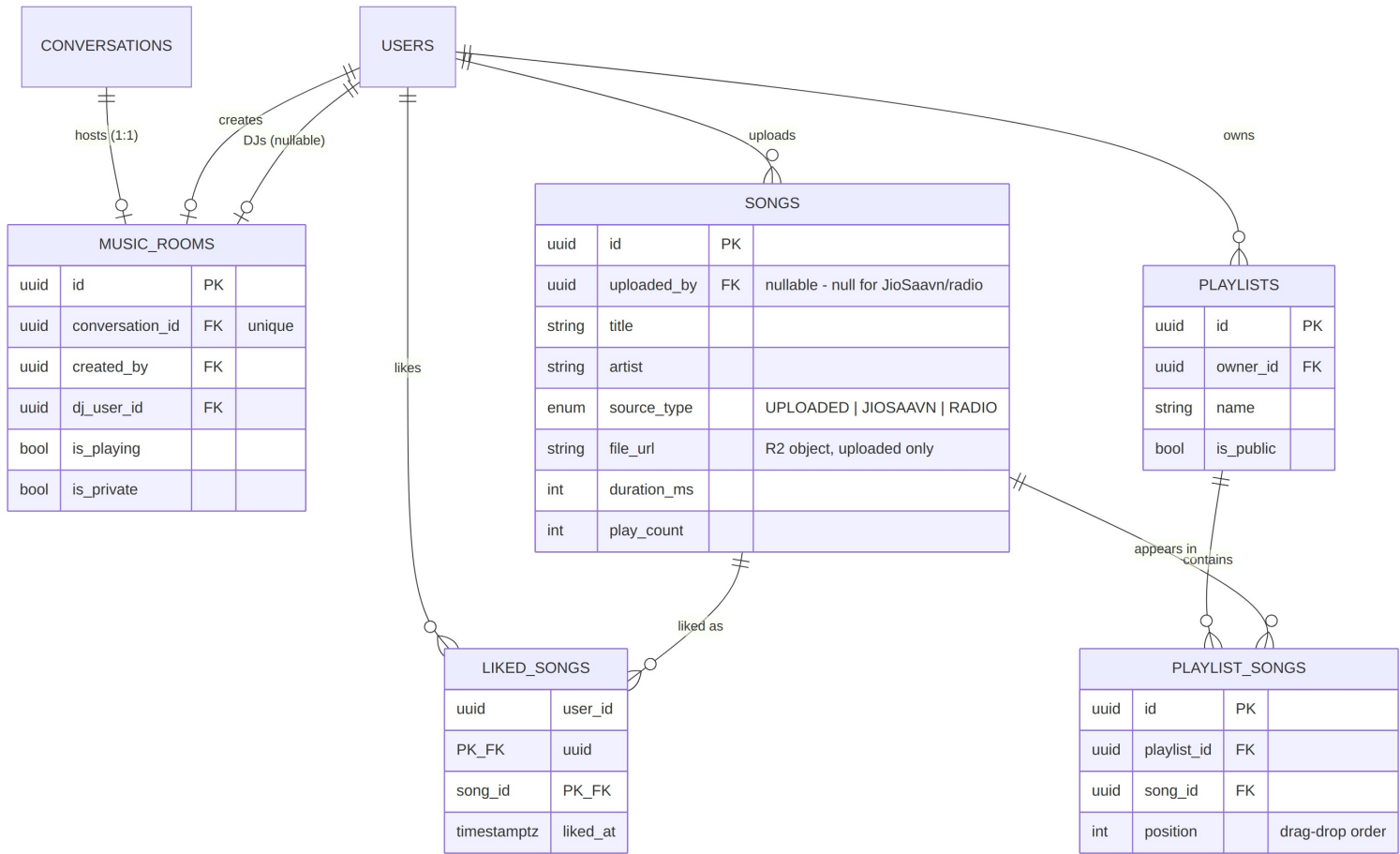


Figure 3 — music domain ERD. `music_rooms` is the join point back to `conversations` (1:1).

**Reading the cardinality:** `conversations ||--o{ conversation_members` means one conversation has zero-or-many members (the standard case). The one exception worth noting is `conversations ||--o| music_rooms` — a strict 1:1, enforced in Prisma by making `conversation_id` a `@unique` foreign key on `music_rooms` rather than giving `MusicRoom` its own independent identity. A conversation either has a room or it doesn't; it can never have two.

### 3. PostgreSQL — full table specifications & design rationale

The PRD's §9.1 gives a skeleton of 9 tables. This section expands each one to production grade and adds four tables the PRD's feature list implies but doesn't yet name explicitly — `devices` , `blocked_users` , `liked_songs` , `notification_preferences` , and `call_logs` — with the reasoning for each addition called out.

#### 3.1 users

| Field                           | Type                             | Notes                                                                                                                                                                                         |
|---------------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id                              | uuid PK                          | v4, generated client-side or by Postgres                                                                                                                                                      |
| username                        | varchar(32)<br>unique            | Telegram-style handle; the identity that never requires a phone number (F-05)                                                                                                                 |
| phone                           | varchar(20)<br>unique, nullable  | Either phone or email required — enforce in application layer, not a DB CHECK, since Prisma can't easily express "at least one of two nullable columns"                                       |
| email                           | varchar(255)<br>unique, nullable | —                                                                                                                                                                                             |
| password_hash                   | text, nullable                   | Null for OTP-only or Google-only accounts. Argon2id, never bcrypt for new code                                                                                                                |
| google_id                       | text unique,<br>nullable         | OAuth subject claim (F-04)                                                                                                                                                                    |
| avatar_url, bio,<br>status_text | text                             | Profile basics                                                                                                                                                                                |
| is_verified                     | boolean                          | OTP or email verification complete                                                                                                                                                            |
| last_seen_at                    | timestampz                       | Written on disconnect, not on every heartbeat — live "online" state is Redis's <code>presence:{user_id}</code> key, this column is only the fallback "last seen 2h ago" once presence expires |
| deleted_at                      | timestampz,<br>nullable          | Soft delete — see §8                                                                                                                                                                          |

#### 3.2 devices NEW

The PRD lists FCM push (F-45) but the skeleton in §9.1 has no table to hold an FCM token. A single `fcmtoken` column on `users` would break the moment someone is logged in on phone + web simultaneously — the second login would silently overwrite the first device's push target and one of the two stops receiving notifications. `devices` is a one-to-many table precisely so multi-device (a stated non-goal-turned-goal once you support both web and, later, React Native) doesn't quietly break push.

| Field          | Type                            | Notes                                                                                                    |
|----------------|---------------------------------|----------------------------------------------------------------------------------------------------------|
| id             | uuid PK                         | —                                                                                                        |
| user_id        | uuid FK → users, cascade delete | —                                                                                                        |
| device_type    | enum: ios   android   web       | —                                                                                                        |
| fcmtoken       | text, nullable                  | Rotates periodically; nullable because a device can exist mid-registration before FCM hands back a token |
| push_enabled   | boolean                         | Per-device mute, distinct from per-conversation mute (§3.9)                                              |
| last_active_at | timestampz                      | Used to prune stale devices/tokens on a cron                                                             |

#### 3.3 user\_contacts

| Field               | Type            | Notes                                               |
|---------------------|-----------------|-----------------------------------------------------|
| id                  | uuid PK         | —                                                   |
| user_id, contact_id | uuid FK → users | Directed edge — A adding B does not imply B added A |
| nickname            | text, nullable  | Local display name override, WhatsApp-style         |
| added_at            | timestampz      | —                                                   |

`@unique([user_id, contact_id])` prevents duplicate contact rows; querying "is B in A's contacts, or A in B's" is the one place the PRD's note to "query symmetrically" applies — the application layer runs an OR across both directions rather than the DB enforcing symmetry.

#### 3.4 blocked\_users NEW

Not in the PRD skeleton, but block is a safety-critical feature every WhatsApp-class product ships from day one, and it has real schema implications: message-send and music-room-invite paths both need to check this table before writing to Mongo/Redis. Modelled identically to `user_contacts` (directed edge, composite unique) for consistency.

3.5 conversations

| Field            | Type                       | Notes                                                                                               |
|------------------|----------------------------|-----------------------------------------------------------------------------------------------------|
| id               | uuid PK                    | —                                                                                                   |
| type             | enum: dm   group   channel | One table for all three, per PRD — behaviour branches on this field in application code, not schema |
| name, avatar_url | text, nullable             | Null for DMs (name is derived from the other member client-side)                                    |
| created_by       | uuid FK → users            | —                                                                                                   |
| created_at       | timestamptz                | —                                                                                                   |

3.6 conversation\_members

| Field                    | Type                         | Notes                                                                                                                                                                        |
|--------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id                       | uuid PK                      | —                                                                                                                                                                            |
| conversation_id, user_id | uuid FK                      | —                                                                                                                                                                            |
| role                     | enum: member   admin   owner | Owner = creator by default; promotable to admin for group management                                                                                                         |
| joined_at                | timestamptz                  | —                                                                                                                                                                            |
| last_read_at             | timestamptz, nullable        | The read-receipt cursor — "unread count" = count of Mongo messages with <code>created_at &gt; last_read_at</code> for that conversation, computed on read rather than stored |

**Hottest index in the system:** `@@unique([conversation_id, user_id])` is checked on every single message send and every socket connection to verify membership before allowing a join to a room's WebSocket namespace. It must stay a tight two-column composite index — resist the temptation to widen it.

3.7 music\_rooms , songs , playlists , playlist\_songs

| Table          | Key fields beyond PRD skeleton                    | Rationale                                                                                                                                                                                                                                                                                                                |
|----------------|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| music_rooms    | —                                                 | is_playing here is a snapshot reconciled from Redis when a room closes, not the live value — the live value is room:{room_id}:state in Redis. Duplicating a boolean snapshot avoids a full-table scan of Redis just to answer "which of my conversations currently have an active room" on the conversation list screen. |
| songs          | uploaded_by nullable                              | Null for jiosaavn / radio source types — those songs aren't "owned" by any Rhynek user                                                                                                                                                                                                                                   |
| playlist_songs | position as plain int, not float/fractional-index | At MVP scale (personal playlists, not shared real-time editing), re-sequencing integers on reorder is simpler than fractional indexing and the write volume doesn't justify the added complexity — worth revisiting only if collaborative real-time playlist editing becomes a feature                                   |

3.8 liked\_songs NEW

F-28 ("liked songs + playlists") implies two distinct concepts that the PRD skeleton collapses into one. A liked song is a boolean-like relationship (you either liked it or didn't, no ordering, no name); a playlist is curated, named, and ordered. Modelling "Liked Songs" as a special auto-created playlist row is a common shortcut that leaks complexity later — you need to special-case that one playlist everywhere (can't rename it, can't delete it, doesn't show a position). A dedicated liked\_songs composite-key table (no surrogate id needed) keeps that logic out of the playlist model.

3.9 notifications & notification\_preferences

notifications matches the PRD skeleton directly. notification\_preferences is new — it's what F-47 ("notification preferences per conversation") needs: a per-user, per-conversation mute flag with an optional muted\_until for "mute for 8 hours" style options, distinct from the device-level push\_enabled kill switch in §3.2.

3.10 call\_logs NEW — Phase 4b

WebRTC signaling itself is transient (handled over Socket.IO, never touches a database), but "recent calls" is a relational, low-write-volume history list that belongs in Postgres — not in Mongo alongside messages, which would force every message-history query to filter out call records it doesn't need.



## 4. MongoDB — messages & stories schema design

### 4.1 messages

```
{
  _id: ObjectId,
  conversation_id: UUID,          // references Postgres conversations.id — no FK, app-enforced
  sender_id: UUID,               // references Postgres users.id
  type: 'text' | 'image' | 'audio' | 'song_share' | 'file',
  content: {
    text: String,
    media_url: String,
    song_id: UUID,               // references Postgres songs.id when type = song_share
    file_name: String,
    file_size: Number
  },
  encrypted_payload: String,     // libsodium ciphertext for E2E text (F-07)
  reply_to: ObjectId | null,     // threaded reply (F-11)
  reactions: [{ user_id: UUID, emoji: String, created_at: Date }],
  read_by: [{ user_id: UUID, read_at: Date }],
  deleted_at: Date | null,       // soft delete for self / for everyone (F-13)
  edited_at: Date | null,        // F-12
  disappear_at: Date | null,     // TTL-index candidate, F-15
  created_at: Date
}
```

**Index:** compound (`conversation_id, created_at DESC`) — this is the PRD's stated highest-write-volume collection and every read is "give me the last N messages in this conversation," so this compound index is what makes that a cheap indexed range scan instead of a collection scan.

**Why embed reactions and read\_by as arrays instead of separate collections:** both are small (bounded by group size, max 256 per PRD §4.1), always read together with the message, and never queried independently ("show me all reactions across all conversations" isn't a real feature). That's exactly the profile document embedding is good at. The one risk — unbounded array growth — doesn't apply here because group size is capped.

**Disappearing messages TTL:** Mongo TTL indexes only auto-delete on a whole-document basis based on a single date field, evaluated roughly once a minute. That's the right mechanism for `disappear_at` — but soft-delete ( `deleted_at` ) and hard TTL-expiry are two different lifecycles and shouldn't be conflated. A "delete for me" action sets `deleted_at` per-user (needs to become an array like `deleted_for: [user_id]` rather than a single timestamp, since delete-for-me is per-viewer, not global) — worth flagging as a schema refinement before Phase 2 ships F-13 fully.

### 4.2 stories NEW — not in PRD skeleton

F-37 (Status/Stories, 24h expiry) needs a home. Options considered: Postgres with a cron sweep, or Mongo with a native TTL index. Mongo wins here for the same reason messages do — it's append-heavy, ephemeral by design, and a TTL index gives automatic expiry with zero application-level cleanup code.

```
{
  _id: ObjectId,
  user_id: UUID,
  media_url: String,
  caption: String | null,
  viewed_by: [{ user_id: UUID, viewed_at: Date }],
  created_at: Date,
  expires_at: Date                // TTL index on this field, expireAfterSeconds: 0
}
```

**Index:** TTL index on `expires_at`, plus a compound (`user_id, created_at DESC`) for "fetch this user's active stories."

## 5. Redis — key design & TTL strategy

This matches and slightly extends PRD §9.3. The organizing principle: **every key pattern has an explicit TTL unless it's a durable pointer that's cheap to overwrite** (room state, room queue) — nothing in Redis should be able to leak memory indefinitely without a lifecycle decision behind it.

| Key pattern               | Type             | TTL            | Purpose                                                                                                     |
|---------------------------|------------------|----------------|-------------------------------------------------------------------------------------------------------------|
| presence:{user_id}        | string           | 30s heartbeat  | Online/away indicator — client re-sends heartbeat to keep alive; absence of the key = offline               |
| room:{room_id}:state      | hash             | none           | song_id, position_ms, is_playing, dj_id, updated_at — the live playback truth                               |
| room:{room_id}:queue      | list             | none           | Upcoming song_ids                                                                                           |
| room:{room_id}:members    | sorted set       | none           | score = join_timestamp — doubles as DJ-election order (F-21): oldest member becomes DJ if current DJ leaves |
| typing:{conv_id}          | set              | 3s auto-expire | Typing indicator                                                                                            |
| session:{user_id}         | string           | 7 days         | Refresh token hash — rotated on use, see §8.4 token-family model                                            |
| ratelimit:{ip}:{endpoint} | string (counter) | 60s            | Per-IP rate limiting                                                                                        |
| otp:{phone}               | hash             | 5 min          | { code, attempts }                                                                                          |

**Room state has no TTL by design** — a room can be idle (paused) indefinitely and it should survive a Redis restart's worth of grace period without losing DJ assignment. The tradeoff: rooms need an explicit close/cleanup path (on last member leaving, or an idle sweep) rather than relying on expiry, otherwise abandoned rooms accumulate forever on the free tier's 500k-commands/day budget (PRD §15 risk).

## 6. Indexing, cascade rules & query-path analysis

| Query path                                                                      | Index used                                     | Cascade behaviour                                           |
|---------------------------------------------------------------------------------|------------------------------------------------|-------------------------------------------------------------|
| "Is user X a member of conversation Y?" (every message send, every socket auth) | conversation_members(conversation_id, user_id) | Cascade delete when conversation or user is deleted         |
| "Give me all conversations for user X" (conversation list)                      | conversation_members(user_id)                  | —                                                           |
| "Give me the last 50 messages in conversation Y" (Mongo)                        | messages(conversation_id, created_at DESC)     | App-level: soft-delete via deleted_at , never a hard remove |
| "Search songs by title" (F-24, F-41)                                            | songs(title)                                   | —                                                           |
| "Notifications for user X, newest first"                                        | notifications(user_id, created_at DESC)        | Cascade delete with user                                    |
| "Recent calls for conversation Y"                                               | call_logs(conversation_id, started_at DESC)    | —                                                           |

**Cascade rule of thumb applied throughout the schema:** child rows that only make sense in the context of their parent (memberships, playlist\_songs, liked\_songs, notification prefs, devices) cascade-delete. `users` itself never hard-deletes (see §8) — so in practice the cascade chain only fires when a conversation or playlist is deliberately deleted, not when a user account is removed.

## 7. Scaling path — from 500 to 25,000 DAU and beyond

| Layer       | MVP (Month 1)                                                 | Growth trigger                                                                           | Next step                                                                                                                                                   |
|-------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL  | Single Supabase instance                                      | Read-heavy endpoints (conversation list, profile lookups) start dominating write latency | Add read replicas — Prisma supports replica routing without schema changes (PRD §14.2)                                                                      |
| MongoDB     | Single Atlas cluster                                          | Message write volume approaches cluster write-capacity ceiling                           | Shard on <code>conversation_id</code> — already the compound index prefix, so it's also a natural shard key that keeps a conversation's messages co-located |
| Redis       | Single Upstash instance (Mumbai)                              | 500k commands/day free-tier ceiling approached (PRD §15)                                 | Upgrade tier first (\$10/mo per PRD) before any architectural change — the room-state model doesn't need to change, just capacity                           |
| Music rooms | Uncapped in schema, 50-member soft cap in product (PRD §14.2) | Room member sorted-set + fan-out socket broadcast cost grows with room size              | Cap enforced at the Socket.IO join handler, not the database — the schema doesn't need to change to raise or lower this number                              |

## 8. GDPR / DPDPA compliance in the schema

---

PRD §14.4 requires export and delete rights. The schema decisions that make this tractable rather than a fire-drill later:

- **Soft delete on users** ( `deleted_at` ) — a hard delete would either cascade-null every message's `sender_id` in Mongo (breaking history for everyone else in the conversation) or require touching every collection at delete time. Soft delete + a scheduled anonymization job (null out `email`, `phone`, `avatar_url`, replace `username` with a placeholder) satisfies "right to be forgotten" without breaking referential integrity for other users' conversation history.
- **Data export** is a read-only join across `users` → `conversation_members` → Mongo `messages` filtered by `sender_id` — no schema change needed, just an application-layer export job.
- **Region:** Supabase, Atlas, and Upstash all provisioned in `ap-south-1` (Mumbai) per PRD §14.4 — a schema-adjacent but infrastructure-level decision worth re-verifying at each provider's console since none of the three encode region in the schema itself.

## 9. Does Prisma auto-generate ER diagrams? — direct answer

**No — not out of the box.** Running `npx prisma generate` or `npx prisma migrate dev` only does what its name says: generates the Prisma Client and applies migrations. It does not produce a visual diagram anywhere in the default toolchain.

What you're picturing is achievable, but it requires adding a **generator** to `schema.prisma` that hooks into the same `prisma generate` command:

```
generator erd {
  provider = "prisma-erd-generator"
  output   = "../docs/ERD.svg"    // or .png, .md (Mermaid), .pdf
}
```

Install with `npm install -D prisma-erd-generator @mermaid-js/mermaid-cli`. Once that generator block exists, every time you run `npx prisma generate` after editing the schema, it will also regenerate an up-to-date ER diagram alongside the client — which is the closest real equivalent to "adding a table automatically draws the diagram." It's not automatic in the sense of watching the file and redrawing live; it redraws on the next generate command, which in practice means every time you save the schema and re-run the command Antigraity or your dev workflow already runs.

| Tool                                                               | How it works                                                                                   | Best for                                                                                                                                                         |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| prisma-erd-generator                                               | Generator plugin, hooks into <code>prisma generate</code> , renders via Mermaid under the hood | Keeping a diagram in your repo that never goes stale relative to the schema — the approach above                                                                 |
| Prisma's own docs generator ( <code>prisma-docs-generator</code> ) | Generates a browsable HTML doc site, includes a relation diagram                               | Living internal documentation rather than a single exportable image                                                                                              |
| dbdiagram.io                                                       | Paste your schema (or a DBML export) in manually, it renders interactively                     | One-off exploration or presenting to non-engineers; not auto-synced                                                                                              |
| This report's diagrams (Figures 1-3)                               | Hand-authored Mermaid, rendered once for this document                                         | A snapshot for R&D discussion — once your actual Prisma schema is finalized, wiring up <code>prisma-erd-generator</code> is the better long-term source of truth |

Recommendation: add the `prisma-erd-generator` block to your real `schema.prisma` as soon as Phase 1 schema work starts. It costs one generator block and two dev-dependencies, and it means the ERD in your repo is provably in sync with the migrations — no separate diagram to remember to update.

## 10. Full `schema.prisma` reference

The complete, ready-to-use schema discussed throughout this document — delivered alongside this PDF as `schema.prisma` . Reproduced here for reference.

```
// =====
// RHYNK – PostgreSQL schema (Prisma)
// Region: ap-south-1 (Mumbai) · Provider: Supabase
// All IDs: UUID v4. All timestamps: UTC (timestampz).
// This is the SYSTEM OF RECORD for identity, structure, and relationships.
// High-write ephemeral data (messages, stories, presence, room playback state)
// intentionally lives OUTSIDE Postgres – see MongoDB / Redis sections of the PRD.
// =====

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// -----
// ENUMS
// -----

enum ConversationType {
  DM
  GROUP
  CHANNEL
}

enum MemberRole {
  MEMBER
  ADMIN
  OWNER
}

enum SongSourceType {
  UPLOADED
  JIOSAAVN
  RADIO
}

enum DeviceType {
  IOS
  ANDROID
  WEB
}

enum NotificationType {
  NEW_MESSAGE
  MENTION
  MUSIC_ROOM_INVITE
  CONTACT_REQUEST
  CALL_MISSED
  SYSTEM
}

enum CallType {
  VOICE
  VIDEO
}

enum CallStatus {
  MISSED
  DECLINED
  COMPLETED
  FAILED
}

// -----
// IDENTITY
// -----

model User {
  id          String   @id @default(uuid()) @db.Uuid
  username    String   @unique @db.VarChar(32)
  phone       String?  @unique @db.VarChar(20)
  email       String?  @unique @db.VarChar(255)
  passwordHash String? @map("password_hash") // null for phone-OTP-only / OAuth-only users
  googleId    String?  @unique @map("google_id")

  avatarUrl   String?  @map("avatar_url")
  bio         String?  @db.VarChar(160)
  statusText  String?  @map("status_text") @db.VarChar(120)
```

```

isVerified Boolean @default(false) @map("is_verified")
lastSeenAt DateTime? @map("last_seen_at")

// GDPR / DPDPA – soft delete, never hard-delete a row with FK fan-out
deletedAt DateTime? @map("deleted_at")

createdAt DateTime @default(now()) @map("created_at")
updatedAt DateTime @updatedAt @map("updated_at")

// Relations
devices Device[]
createdConversations Conversation[] @relation("ConversationCreator")
memberships ConversationMember[]
createdRooms MusicRoom[] @relation("RoomCreator")
djRooms MusicRoom[] @relation("RoomDJ")
uploadedSongs Song[]
playlists Playlist[]
likedSongs LikedSong[]
contactsOf UserContact[] @relation("ContactOwner")
contactedBy UserContact[] @relation("ContactTarget")
blockedUsers BlockedUser[] @relation("Blocker")
blockedBy BlockedUser[] @relation("Blocked")
notifications Notification[]
notificationPrefs NotificationPreference[]
callsInitiated CallLog[] @relation("CallInitiator")

@@index([phone])
@@index([email])
@@map("users")
}

// Multi-device support: one user, many logged-in devices.
// FCM tokens live here, NOT on User – a user can have a phone + web + tablet
// session simultaneously, each needing its own push target and refresh-token
// lineage (rotation / revocation is per-device, per PRD $12.1 token family model).
model Device {
  id String @id @default(uuid()) @db.Uuid
  userId String @map("user_id") @db.Uuid
  deviceType DeviceType @map("device_type")
  fcmToken String? @map("fcm_token")
  pushEnabled Boolean @default(true) @map("push_enabled")
  lastActiveAt DateTime @default(now()) @map("last_active_at")
  createdAt DateTime @default(now()) @map("created_at")

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("devices")
}

model UserContact {
  id String @id @default(uuid()) @db.Uuid
  userId String @map("user_id") @db.Uuid
  contactId String @map("contact_id") @db.Uuid
  nickname String?
  addedAt DateTime @default(now()) @map("added_at")

  owner User @relation("ContactOwner", fields: [userId], references: [id], onDelete: Cascade)
  target User @relation("ContactTarget", fields: [contactId], references: [id], onDelete: Cascade)

  // Directed edge, as PRD notes – query symmetrically (WHERE user_id = X OR contact_id = X)
  @@unique([userId, contactId])
  @@index([contactId])
  @@map("user_contacts")
}

model BlockedUser {
  id String @id @default(uuid()) @db.Uuid
  blockerId String @map("blocker_id") @db.Uuid
  blockedId String @map("blocked_id") @db.Uuid
  createdAt DateTime @default(now()) @map("created_at")

  blocker User @relation("Blocker", fields: [blockerId], references: [id], onDelete: Cascade)
  blocked User @relation("Blocked", fields: [blockedId], references: [id], onDelete: Cascade)

  @@unique([blockerId, blockedId])
  @@map("blocked_users")
}

// -----
// CONVERSATIONS & MEMBERSHIP
// -----

model Conversation {
  id String @id @default(uuid()) @db.Uuid
  type ConversationType
  name String? // null for DMs, required for GROUP/CHANNEL
  avatarUrl String? @map("avatar_url")

```



```

    createdBy String      @map("created_by") @db.Uuid
    createdAt DateTime    @default(now()) @map("created_at")

    creator User          @relation("ConversationCreator", fields: [createdBy], references: [id])
    members ConversationMember[]
    room MusicRoom?
    notificationPrefs NotificationPreference[]

    @@index([type])
    @@map("conversations")
}

model ConversationMember {
    id String @id @default(uuid()) @db.Uuid
    conversationId String @map("conversation_id") @db.Uuid
    userId String @map("user_id") @db.Uuid
    role MemberRole @default(MEMBER)
    joinedAt DateTime @default(now()) @map("joined_at")
    lastReadAt DateTime? @map("last_read_at") // drives unread-count + read-receipt cursor

    conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)

    // Composite unique – a user can only be in a conversation once.
    // This is also the index Postgres uses to answer "is user X in conversation Y"
    // on every single message-send / socket auth check, so it must stay tight.
    @@unique([conversationId, userId])
    @@index([userId])
    @@map("conversation_members")
}

// -----
// MUSIC
// -----

// One row per conversation that has ever started a room (1:1 with Conversation).
// LIVE playback state (position_ms, is_playing, queue) is intentionally NOT here –
// it lives in Redis because it mutates multiple times per second per active room
// and Postgres should never see that write volume. This table is the durable
// "room exists / who owns it" record; Redis is the hot path.
model MusicRoom {
    id String @id @default(uuid()) @db.Uuid
    conversationId String @unique @map("conversation_id") @db.Uuid
    createdBy String @map("created_by") @db.Uuid
    djUserId String? @map("dj_user_id") @db.Uuid
    isPlaying Boolean @default(false) @map("is_playing") // last-known snapshot, reconciled from Redis on room close
    isPrivate Boolean @default(false) @map("is_private")
    createdAt DateTime @default(now()) @map("created_at")

    conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)
    creator User @relation("RoomCreator", fields: [createdBy], references: [id])
    dj User? @relation("RoomDJ", fields: [djUserId], references: [id])

    @@map("music_rooms")
}

model Song {
    id String @id @default(uuid()) @db.Uuid
    uploadedBy String? @map("uploaded_by") @db.Uuid // null for jiosaavn/radio sourced songs
    title String
    artist String
    album String?
    sourceType SongSourceType @map("source_type")
    sourceUrl String? @map("source_url") // external JioSaavn/radio stream reference
    fileUrl String? @map("file_url") // Cloudflare R2 object URL for uploaded songs
    durationMs Int @map("duration_ms")
    coverUrl String? @map("cover_url")
    playCount Int @default(0) @map("play_count")
    createdAt DateTime @default(now()) @map("created_at")

    uploader User? @relation(fields: [uploadedBy], references: [id])
    playlistSongs PlaylistSong[]
    likedBy LikedSong[]

    @@index([sourceType])
    @@index([title])
    @@map("songs")
}

model Playlist {
    id String @id @default(uuid()) @db.Uuid
    ownerId String @map("owner_id") @db.Uuid
    name String
    isPublic Boolean @default(false) @map("is_public")
    createdAt DateTime @default(now()) @map("created_at")

    owner User @relation(fields: [ownerId], references: [id], onDelete: Cascade)
    songs PlaylistSong[]

```

```

    @@index([ownerId])
    @map("playlists")
}

model PlaylistSong {
    id String @id @default(uuid()) @db.Uuid
    playlistId String @map("playlist_id") @db.Uuid
    songId String @map("song_id") @db.Uuid
    position Int // drag-drop ordering; re-sequenced on reorder, not a dense array

    playlist Playlist @relation(fields: [playlistId], references: [id], onDelete: Cascade)
    song Song @relation(fields: [songId], references: [id], onDelete: Cascade)

    @@unique([playlistId, songId])
    @@index([playlistId, position])
    @map("playlist_songs")
}

// "Liked songs" is its own join table rather than a special playlist row –
// keeps the semantics (like vs. curated playlist) and query patterns separate.
model LikedSong {
    userId String @map("user_id") @db.Uuid
    songId String @map("song_id") @db.Uuid
    likedAt DateTime @default(now()) @map("liked_at")

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    song Song @relation(fields: [songId], references: [id], onDelete: Cascade)

    @@id([userId, songId])
    @map("liked_songs")
}

// -----
// NOTIFICATIONS
// -----

model Notification {
    id String @id @default(uuid()) @db.Uuid
    userId String @map("user_id") @db.Uuid
    type NotificationType
    title String
    body String
    data Json? // deep-link payload: conversation_id, room_id, etc.
    readAt DateTime? @map("read_at")
    createdAt DateTime @default(now()) @map("created_at")

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)

    @@index([userId, createdAt(sort: Desc)])
    @map("notifications")
}

// Per-conversation mute / notification-level preference (F-47).
model NotificationPreference {
    userId String @map("user_id") @db.Uuid
    conversationId String @map("conversation_id") @db.Uuid
    muted Boolean @default(false)
    mutedUntil DateTime? @map("muted_until") // null + muted=true => muted indefinitely

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)

    @@id([userId, conversationId])
    @map("notification_preferences")
}

// -----
// CALLS (Phase 4b)
// -----

// WebRTC signaling itself is ephemeral (Socket.IO), but the CALL RECORD –
// who called whom, duration, outcome – is structured history that belongs
// in Postgres, not Mongo (low write volume, needs relational joins for
// "recent calls" screens).
model CallLog {
    id String @id @default(uuid()) @db.Uuid
    conversationId String @map("conversation_id") @db.Uuid
    initiatedBy String @map("initiated_by") @db.Uuid
    callType CallType @map("call_type")
    status CallStatus
    startedAt DateTime @map("started_at")
    endedAt DateTime? @map("ended_at")
    durationSec Int? @map("duration_sec")

    initiator User @relation("CallInitiator", fields: [initiatedBy], references: [id])

    @@index([conversationId, startedAt(sort: Desc)])
    @map("call_logs")
}

```

